

TP Typescript

Utilisez le script <https://www.typescriptlang.org/play> pour rédiger votre code typescript en ligne

Exercice 0) Interface

Ecrire une interface qui représente une personne, sachant qu'une personne a les caractéristiques suivantes :

- son nom
- son age
- détenteur du permis de conduire (oui/non)

Créer un objet ayant pour type cette interface et afficher le dans la console grâce au code suivante :

```
console.log(votreObject)
```

Exercice 1) Un pour toi, un pour moi

Ecrire une fonction **pourToiPourMoi** qui prends un nom en paramètre et retourne la phrase suivante:

Un pour X, un pour moi
Ou X est le nom donné en paramètre

En revanche, si le paramètre est manquant, la fonction doit renvoyer:

Un pour toi, un pour moi

Exemples:

pourToiPourMoi('toto') => renvoi Un pour toto, un pour moi
pourToiPourMoi() => renvoi Un pour toi, un pour moi

Exercice 2) Bob le chatbot

Ecrire une fonction **tellBob** qui prend en parametre une phrase et qui renvoie les phrases suivantes en fonction de la phrase d'entrée:

"Bien sur" si vous lui posez une question (ie: la phrase termine par un ?)

'Du calme' si vous lui criez dessus (ie la phrase ne contient que des majuscules).

'Ok, si tu veux' si la phrase que vous lui envoyez est vide

'Peu m'importe.' pour tout le reste

Vous pourrez trouver des méthodes bien utiles sur la documentation des chaines de caractères :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Objets_globaux/String

Exercice 3) Pangramme

Un pangramme (du grec ancien πᾶς / pās (« tout ») et γράμμα / gramma (« lettre »)) est une phrase comportant toutes les lettres de l'alphabet. En français, un pangramme comporte au moins 26 lettres.

écrire une fonction **isPangram** qui prend en paramètre une phrase et renvoie vrai ou faux si c'est un pangramme

Si ca peut vous aider, voici les lettres de l'alphabets dans une variable :

```
const alphabet = "abcdefghijklmnopqrstuvwxyz"
```

Pour tester votre méthode, vous pouvez essayer avec ce pangram : portez ce vieux whisky au juge blond qui fume

Exercice 4)Les allergies

Etant donné un score allergique d'une personne, vous devez déterminer si une personne est allergique à certaines choses et pouvoir donner la liste de toutes les choses auxquelles elle est allergique.

Un test allergique est une valeur numérique qui reprend les valeurs des scores des choses auxquels la personne est allergique.

Voici une liste d'objet avec leur score allergique

- oeuf (1)
- cacahuète (2)

- fruit de mer (4)
- fraise (8)
- tomate (16)
- chocolat (32)
- pollen (64)
- chats (128)

Donc si Bob est allergique aux cacahuètes et au chocolat, il a un score allergique de 34

Donc depuis un score de 34, votre programme est capable de

- Dire si la personne est allergique à un des objets listés ci dessus.
- Dire quels sont les objets auxquels la personne est allergique.

Ecrivez une classe **Patient**, dont le constructeur prend en paramètre le score allergique du patient.

Cette classe devra comporter 2 méthodes:

isAllergicTo qui prend un paramètre un aliment et renvoie si oui ou non la personne est allergique à cet objet

listAllergies: renvoie la liste des objets auxquels le patient est allergique

Exemple pour tester votre code :

```
const patient: Patient = new Patient(34);
```

```
patient.listAllergies() => renvoie ['chocolat', 'cacahuete']
```

```
patient.isAllergicTo("chat") => renvoie false
```

```
patient.isAllergicTo("cacahuete") => renvoie true
```

Exercice 6 (Bonus))

Instructions

Implement a doubly linked list.

Like an array, a linked list is a simple linear data structure. Several common data types can be implemented using linked lists, like queues, stacks, and associative arrays.

A linked list is a collection of data elements called nodes. In a singly linked list each node holds a value and a link to the next node. In a doubly linked list each node also holds a link to the previous node.

You will write an implementation of a doubly linked list. Implement a Node to hold a value and pointers to the next and previous nodes. Then implement a List which holds references to the first and last node and offers an array-like interface for adding and removing items:

push (insert value at back);
pop (remove value at back);
shift (remove value at front).
unshift (insert value at front);

To keep your implementation simple, the tests will not cover error conditions. Specifically: pop or shift will never be called on an empty list.

If you want to know more about linked lists, check [Wikipedia](#).